Compte Rendu : Projet Jeu de Bataille Navale

Jihene JAZIRI & Leo BOURGET

19 avril 2026

Table des matières

1 Introduction **3**

2 Fonctionnalités implémentées **3**
2.1 Interfaces et Navigation 3
2.2 Système de Pilotage (Commandes) 3
2.3 Objectifs du Jeu 3

3 Choix d'architecture **4**
3.1 Organisation des classes 4
3.2 Le point d'entrée : Main 4

4 Difficultés rencontrées et solutions **4**

5 Répartition des tâches et retour d'expérience **5**
5.1 Démarche et apprentissage 5
5.2 Répartition du travail 5
5.3 Retour d'expérience 5

6 Conclusion **6**

1 Introduction

Ce rapport présente le développement de notre jeu de bateau réalisé en C++. L'objectif était de concevoir une application robuste en appliquant les principes de la programmation orientée objet.

Le concept du jeu consiste à piloter un navire afin de récupérer des objets dispersés dans l'eau. Le joueur est soumis à une contrainte de temps stricte : il doit collecter l'intégralité des objets avant la fin du chronomètre. Si le temps s'écoule alors qu'il reste encore des éléments en mer, la partie est perdue ; à l'inverse, si tous les objets sont récupérés dans le délai imparti, la victoire est remportée.

2 Fonctionnalités implémentées

Le jeu est structuré autour de plusieurs interfaces et d'un système de pilotage précis. Nous avons implémenté les éléments suivants :

2.1 Interfaces et Navigation

- **Menu Principal :** Une page d'accueil permettant de lancer la partie ou d'accéder aux régles du jeu .
- **Page des Paramètres :** Une interface dédiée à l'explication du jeu.
- **Écran de Jeu :** La zone de simulation où le joueur pilote le navire.
- **Gestion de fin de partie :** Des écrans de Victoire et de Défaite s'affichent selon le résultat. Ces écrans incluent une option pour retourner au menu principal et relancer une partie.
- **Adaptabilité :** La fenêtre de jeu est entièrement redimensionnable pour s'adapter à l'écran de l'utilisateur.

2.2 Système de Pilotage (Commandes)

Le contrôle du bateau repose sur une gestion réaliste de la propulsion et de la direction via le clavier :

- **Flèche Haut :** Augmenter le régime moteur (RPM) pour accélérer.
- **Flèche Bas :** Diminuer le régime moteur (RPM) pour ralentir ou reculer.
- **Flèche Droite :** Appliquer une rotation du navire vers la droite.
- **Flèche Gauche :** Appliquer une rotation du navire vers la gauche.

2.3 Objectifs du Jeu

- Système de détection de collision pour le ramassage des objets.
- Gestion du compte à rebours (Timer) dictant la condition de victoire ou de défaite.
- Gestion du score tout au long de la partie.

3 Choix d'architecture

L'agencement du code suit une logique orientée objet structurée par "états" de fenêtres. Ce choix permet d'assurer une séparation claire entre les différentes phases du programme (navigation et jeu).

3.1 Organisation des classes

Nous avons choisi de dédier une classe spécifique à chaque interface du jeu pour faciliter la gestion du rendu et des événements :

Classe Menu : Cette classe regroupe l'ensemble des éléments graphiques de l'écran d'accueil. Elle gère l'affichage du titre et les boutons de navigation .

Classe Paramètre : Elle sert de support à l'explication des règles du jeu. Elle permet de présenter au joueur les objectifs et les commandes avant de lancer la partie.

Classe Jeu : C'est le cœur du projet. Elle gère la simulation proprement dite, notamment l'affichage des entités, la mise à jour de la physique et l'affichage des résultats (victoire ou défaite).

3.2 Le point d'entrée : Main

Le fichier `main.cpp` fait office de chef d'orchestre. Il n'est pas seulement le point de départ, il assure la cohérence globale du logiciel en prenant en charge :

- **La boucle principale :** Il maintient le programme actif et gère la cadence de rafraîchissement.
- **La gestion des fenêtres :** Il orchestre le passage d'une classe à l'autre (par exemple, basculer du Menu vers le Jeu).
- **Les interactions :** Il capte les clics de souris et les entrées clavier pour les transmettre aux classes concernées.
- **Mise à jour (Update) :** Il commande la mise à jour systématique de la logique de jeu et du rendu graphique.

4 Difficultés rencontrées et solutions

Au cours du développement, nous avons dû faire face à plusieurs défis techniques, principalement liés à la prise en main de nouvelles bibliothèques et à la gestion de l'affichage dynamique.

Difficulté	Solution mise en œuvre
Apprentissage et compréhension de la bibliothèque SFML.	Étude approfondie de la documentation officielle et réalisation de prototypes simples pour tester le rendu et les événements.
Gestion du redimensionnement de la fenêtre maintien du ratio d'aspect et de la disposition des éléments (fenêtre du jeu).	Utilisation des `sf::View` pour adapter la vue de la caméra et gestion de l'événement `sf::Event::Resized` pour recalculer les positions.

Table 1 – Tableau des problématiques techniques rencontrées

5 Répartition des tâches et retour d'expérience

5.1 Démarche et apprentissage

Le projet a débuté par une phase d'appropriation des outils technologiques. Nous avons d'abord concentré nos efforts sur la compréhension des bibliothèques **SFML** (pour le rendu graphique) et **Libphysics** (pour la simulation physique).

Notre démarche a été progressive :

- **Première phase :** Création d'objets simples et mise en place de leurs premiers mouvements à l'écran.
- **Deuxième phase :** Intégration des forces physiques pour obtenir un comportement réaliste des entités, conformément au cahier des charges du projet.

5.2 Répartition du travail

Afin d'optimiser le développement, nous avons séparé les responsabilités selon deux axes principaux :

Jihene JAZIRI : S'est concentrée sur l'environnement **SFML**. Elle a conçu et implémenté toute la dimension visuelle du projet, notamment :
- La création et la gestion du menu principal.
- L'interface des paramètres.
- L'affichage global du jeu et l'intégration des assets graphiques.

Léo BOURGET : S'est spécialisé dans la partie **physique et logique de jeu**. Il a eu la charge de :
- La gestion dynamique du bateau et de ses mouvements.
- La génération des obstacles dans l'environnement marin.
- Le système de détection et de ramassage des objets par le navire.

5.3 Retour d'expérience

Ce projet a été une expérience très positive pour notre groupe. Nous avons particulièrement apprécié le concept du jeu, qui offrait un défi de programmation stimulant et concret.

Sur le plan humain, la collaboration au sein du binôme a été excellente. La communication fluide nous a permis de surmonter les obstacles techniques et de progresser efficacement. Cependant, le principal défi a été la gestion du temps : nous avions de nombreuses idées et options supplémentaires pour rendre le jeu plus complet et développé, mais le délai imparti nous a contraints à nous concentrer sur les fonctionnalités essentielles.

Le travail en groupe sur ce projet C++ nous a notamment permis de :

— **Développer une cohésion d'équipe :** Apprendre à coordonner nos efforts pour fusionner nos modules respectifs (physique et graphique).

— **Approfondir nos compétences techniques :** Maîtriser l'utilisation de bibliothèques externes comme SFML et Libphysics.

— **Gérer les priorités :** Faire des choix de conception pour livrer une version stable malgré les contraintes de temps.

6 Conclusion

La principale réussite de ce travail réside dans la création d'une application complète, allant de la gestion physique des mouvements du bateau à la mise en place d'une interface graphique fluide avec SFML, et surtout la mise en pratique approfondie du langage C++. Malgré les défis techniques rencontrés, nous avons réussi à livrer un logiciel fonctionnel respectant l'ensemble des conditions imposées par le cahier des charges.

Ce projet nous a également sensibilisés à la réalité du développement en équipe : savoir se répartir les tâches, communiquer sur les points de blocage et gérer un calendrier serré. Si nous avions disposé de plus de temps, nous aurions souhaité enrichir l'expérience utilisateur par l'ajout de niveaux de difficulté progressifs, de nouveaux types d'obstacles et d'une esthétique sonore plus immersive.

En somme, ce projet fut une étape formatrice et stimulante de notre cursus, renforçant notre envie d'explorer davantage les possibilités offertes par le développement de simulations interactives.